Department of Electrical & Electronic Engineering
Imperial College London

2nd Year Laboratory

Experiment VERI: FPGA Design with Verilog (Part 2)

(webpage: www.ee.ic.ac.uk/pcheung/teaching/E2_Experiment /)

PART 2 – Counters and FSMs

1.0 Learning Outcomes

Part 2 of VERI teaches you:

- how to design different types of counters and timers;
- how to use the Modelsim simulator to verify the correct function of your design and the use of testbenches;
- how to predict the maximum operating clock frequency of your circuit sequential circuits;
- how to design some useful timing and counting components for later part of Experiment VERI.

1.1 Experiment 5: Designing a Counter

Step 1: Create the project for an 8-bit counter

- Create in your directory a folder named part_2.
- Click **file>New Project Wizard**, and create project **ex5** and top level file **ex5_top**. Then click **Finish**.
- Create the Verilog file: "**counter_8.v**" which contains your design in Verilog. I suggest you use convention of using "**_n**" to indicate the number of bits in a module.
- Click **File > New ...** and select Verilog as the new file. An edit window will appear.

Step 2: Enter the Verilog specification of the 8-bit binary counter

- Enter the Verilog module as shown below (next page). Although you can miss out the comments, I recommend that you to retain them because the code is deliberately verbose in order to explain the meaning of the Verilog language.
- The line **`timescale 1ns / 100ps** tells the system to use 1 ns as the unit time step with a time resolution of 100ps.
- Make sure that you fully understand this Verilog code before proceeding to the next step. Save the file as **counter_8.v**. (I recommend that you use module name as the file name to avoid confusion.)

Step 3: Enter the Verilog specification of the 8-bit binary counter

- While is opened in the Editor window, click **Project > Add Current File to** Project, then click **Project > Set as Top Level Entity**. This command tells Quartus that this module is the top-level of your design.

> Normally we use ..._top.v as the top-level module, which connects to physical pins of the FPGA. However, for this experiment, the counter module is verified through simulation. So we don't need to create pin connects. The "Set as Top Level Entity" is very useful if you want to use the simulator to verify different modules in a large design. You can move up and down the module hierarchy and verify them from the lowest level up.

- Click **Processing > Analyze Current File.** This is the fastest way to check if this .v file has any syntax error.
- Then Click **Processing > Start > Start Analysis and Synthesis.** This takes the current Verilog module (and all other modules that it uses if any), and produce a register-level model of your design ready for register-transfer level (RTL) simulation. Unlike full compilation, this step does not require pin assignment and other device specific steps, but is sufficient for you to simulate the circuit as specified in Verilog.

Verilog code: 8-bit counter
(Note that the first character on line 1 before 'timescale' is a backquote ` - not easy to find on many keyboards!)

```
`timescale 1ns / 100ps      // unit time is 1ns, resolution 100ps
//-------------------------------------
// Design Name: counter_8
// Function : an 8-bit synchronous counter with enable input
//-------------------------------------

module counter_8 (
  clock,        // clock input
  enable,       // high enable counting
  count         // count value
);

//-------- Declare ports ---------

  parameter  BIT_SZ = 8;
  input  clock;
  input  enable;
  output [BIT_SZ-1:0]  count;

// count needs to be declared as reg
  reg [BIT_SZ-1:0]  count;

//---- always initialise storage elements such as D-FF
  initial count = 0;

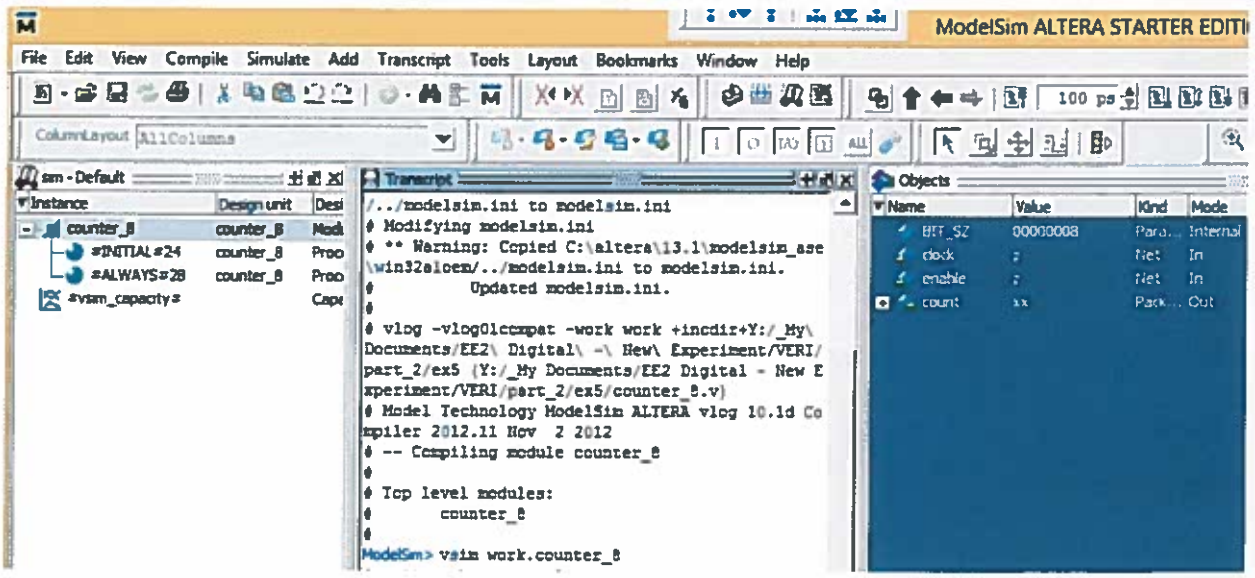
//---- Main body of the module --------

  always @ (posedge clock)
      if (enable == 1'b1)
          count <= count + 1'b1;

endmodule   // end of module
```

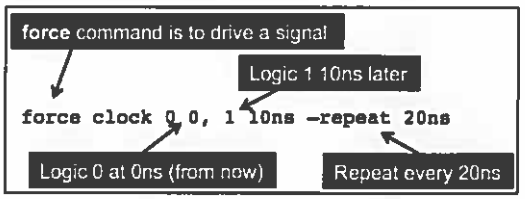
Step 4: Simulate the binary counter

- Click **Tools > Run Simulation Tools > RTL Simulation.** This command starts up Modelsim simulator programme as a separate process. Now you have entered the Modelsim environment.
- Click **Simulate > Start Simulation** .... Then select **work -> counter_8** from the popup window. This tells Modelsim to simulate this module.
- Note that Modelsim provides a number of windowpanes. The most important is the Transcript pane – this is where you enter commands[1] to drive the simulator. The wave pane is where results are displayed as waveforms. You are recommended to un-dock this pane as shown below so that it is in a separate window and spans the whole width of your monitor. Finally, there is the object pane, which shows all the signals (objects) of your design.

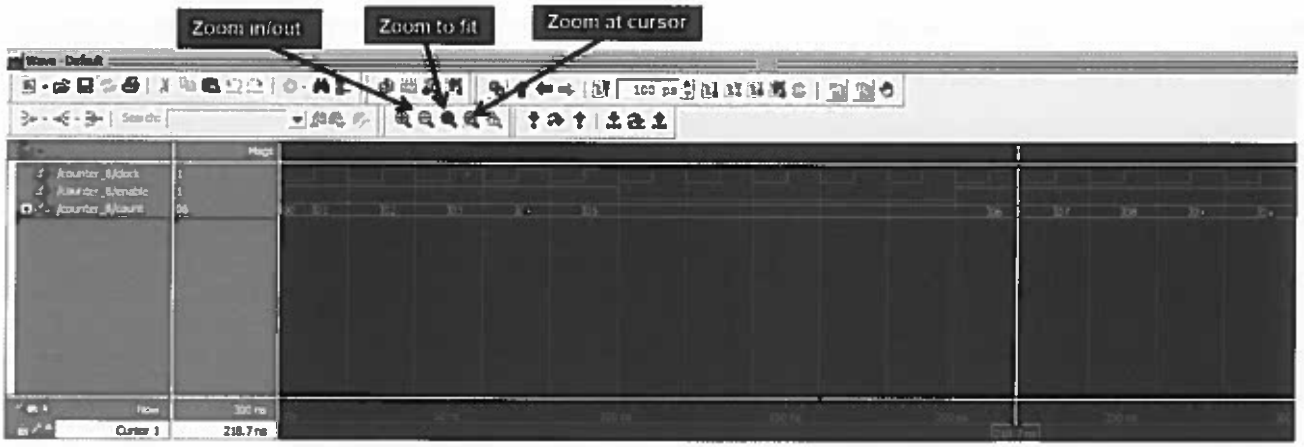


[1] Modelsim uses a scripting language known as **Tcl** in order to control how it is driven. You only need to learn Tcl if you want to do advance stuff with Modelsim for your personal interest.

Step 5: Add waveforms to the Wave window and drive signals



- In the transcript window, enter two commands: "**add wave clock enable**" and "**add wave –hexadecimal count**". This will add these signals as waveforms in the wave pane and show count values as hexadecimal.
- Now we want to drive **clock** with a 50MHz symmetrical signal. To do this, enter:
- Enter: "force enable 1" to enable the counter.
- Enter: "run 100ns" to run the simulator for 5 clock cycles (5 x 20ns = 100ns).
- You will see the waveform pane showing the counter counting from 0 to 5. Now force enable low and run for another 100ns. Then high again and run for 100ns.



- Click on the waveform put a cursor at a specific time for inspecting the signal values. The icons above the waveforms (as labeled) allow you to zoom in and out of the waveform. Try this yourself.

Step 6: Create a Testbench as a DO-file

- Interactively specifying the driving signals is very tedious and prone to error. Therefore the preferred method is to create a **"do"** file which is a text file containing a sequence of commands (as you have previously entered in the transcript window).
- Click **File > new > source** and select new **"do"** file. Then enter the command lines as shown on the right. Then save this as **"tb_counter.do"**.

ment/VERI/part_2/ex5/simulation/modelsim/tb_counter.do - Default

```
Ln#
 1   add wave clock enable
 2   add wave -hexadecimal count
 3   force clock 0 0, 1 10ns -repeat 20ns
 4   force enable 1
 5   run 100ns
 6   force enable 0
 7   run 100ns
 8   force enable 1
 9   run 1000ns
10
```

- Delete all signals from the wave window, and enter command
  vsim> restart
  vsim > do ./tb_counter.do

- This should provide exactly the same waveform results as in step 5. However, the **.do** file can be reused and modified far easier than typing them into the transcript window. It acts as a simple form of a **test-harness** (or **testbench**) for your design. Generally speaking, you must produce testbenches for all your designs instead of using interactive means to test your circuit. Not only because this saves time, it also allows you to change the code and verify its correctness in the same way for each version of your design.

Step 7: Single stepping

- Modelsim is very powerful. You can use it to debug your Verilog design almost like software. However, do remember that we are dealing with a hardware description that operates in parallel. In contrast, software codes are generally procedural and operate sequentially.
- Try the **vsim> step** command or click on the step-command pane  to watch how you can step through your Verilog code. Signal values in the object and the wave windows are updated accordingly.
- Modelsim has many useful features to help you debug your design. Details of all the commands can be found in the Modelsim Reference Manual. This is easily available under **Help > PDF Documentations > Reference Manual**. Beware that this manual is very thick! DO NOT print this out.

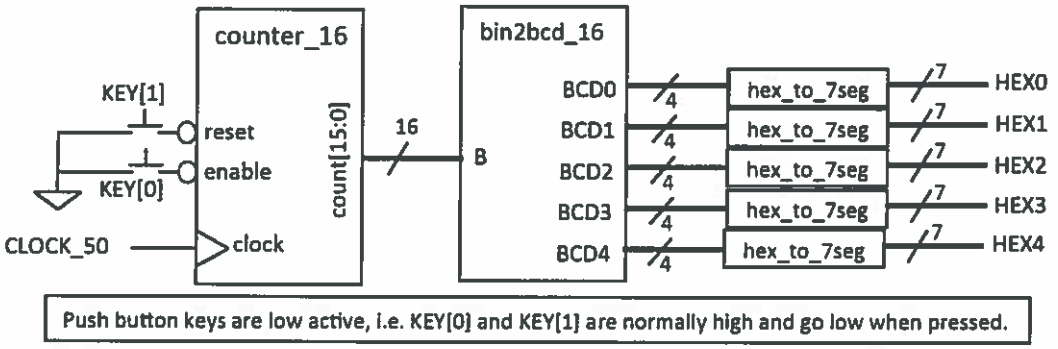
2.0 Experiment 6: Implementing a 16-bit counter on DE1

In this part of the experiment, you will test your counter design on the DE1 board. You will also learn how to find the maximum clock frequency that your design will work correctly.

Step 1: Create a new project ex6, and copy to this directly your files **counter_8.v**. Modify **counter_8.v** to **counter_16.v** and make it a 16-bit counter. Furthermore, add a reset input to reset the count value to zero synchronously to the clock. Download from the experiment webpage the component bin2bcd_16.v, a module I have designed to convert a 16-bit binary number to 5 BCD digits. You will also need the add3_ge5.v module. Put these module in the ../mylib folder, which should also contained the hex_to_7seg.v you designed in Part 1.

Step 2: Create a top-level module ex6_top.v in Verilog to specify the circuit shown below. Make sure that you have added all the relevant Verilog modules to the project using **Project**

> **Add/Remove Files in Project: counter_16.v, ex6_top.v** and finally add **hex_to_7seg.v, add3_ge5.v** and **bin2bcd_16.v** from your library folder **../mylib/**. Go to the **ex6_top.v** window and set this file as your top-level module.



Step 3: Use **Processing > Analyze Current File** check your newly create Verilog files. This is the quickest way to find the basic syntax errors in your Verilog code. Once all the simple errors are fixed, use **Processing > Start Analysis and Elaboration** to perform fuller check of the "**ex6_top.v**" to make sure that files are consistent and correct. There is no need to simulate this circuit.

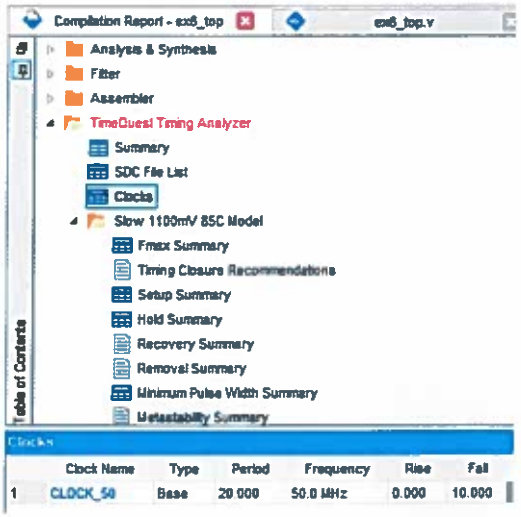
Step 4: Selecting the FPGA Device – Click **Assignments > Device....** and select the correct Cyclone V FPGA: 5CSEMA5F31C6.

Step 5: Pin Assignment – Open the **ex6_top.qsf** file. Examine its content. You will find that no pins are being assigned yet. Insert into this file all the pin assignments. The easiest way to do this is click on: **Edit > Insert file ..** then select **../pin_assignment.txt** (you should have downloaded this file from the Experiment webpage). Note that you are currently not using all the pins assigned in the **pin_assignment.txt** file. Don't worry. This will only produce a few more warning messages. Full compilation can still go ahead without errors.

Step 6: Set clock frequency – Create a new file "**ex6_top.sdc**"[2] which should contain one single line:

```
create_clock -name "CLOCK_50" -period 20.000ns [get_ports {CLOCK_50}]
```

With this, Quartus will know that the signal CLOCK_50 is a 50 MHz clock.



Step 7: Full Compilation – Click: **Processing > Start Compilation**. This will go through the entire compilation process. Examine the Tasks window on the left and see all the steps being taken in order to generate the final bit-stream.

Step 8: Maximum clock frequency – As part of the compilation process, **TimeQuest** timing analyzer is used to predict various timing information. In the "Compilation Report" window, you should see a list of reports resulting from the compilation. Double-click **TimeQuest Timing Analyzer** entry, and you should see a list similar to the one

[2] Synopsis Delay Constraint (.sdc) files are standard formatted files introduced by Synopsis, a well-known company specializing on IC design CAD tools. With this, a designer can specify various timing constraints for the CAD tools the check against. Here we are only using this to define clock frequency.

shown here. Clicking on various entries under this will show the various timing specifications. Answer the following questions:

What are the predicted maximum frequencies for this circuit under the highest and lowest temperatures? What are the other interesting timing data that you can discover with these reports? Why is the TimeQuest entry red, indicating that there may be a problem?

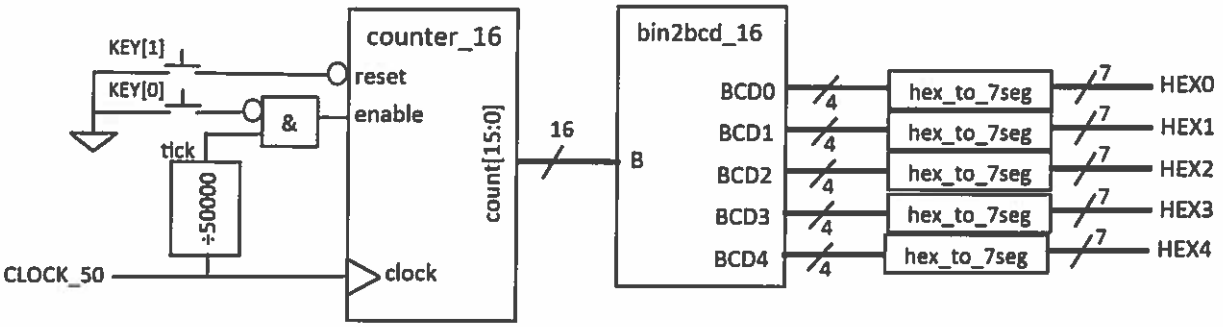
Step 9: Test your design on DE1 – program the DE1 and check that your design works.

Step 10: Examine the amount of FPGA resources being used by this 16-bit counter. Explain the results.

Test-yourself Task (compulsory) **– Cascade counter**

You are now required to create something yourself. In the previous exercise, the 16-bit counter is counting a 20MHz clock. This is much too fast for us to see the counter changing. This part of the experiment requires you use the counter to count the number of millisecond elapsed. You would need to do this by having two counters cascaded (i.e. connected in series) with each other. The overall block diagram is shown below.

The divide-by-50000 circuit generates a 1 cycle high pulse every 50,000 clock cycles. Therefore the output signal tick provides one enable pulse every millisecond. (See notes.)



Modify your circuit to implement this and test the new circuit on the DE1 board.

3.0 Experiment 7: Linear Feedback Shift Register (LFSR) and PRBS

You encountered a 4-bit LFSR in Lecture 5 slide 17, which implements the primitive polynomial: $1 + X^3 + X^4$. You are now required to implement a 7-bit LFSR implementing the polynomial: $1 + X + X^7$. Assuming that you initialize the shift register to 7'd1, work out manually the first 10 sequence values of the output sequence. (The output sequence should be 127 long without repetition, is known as a **pseudo-random binary sequence** or **PRBS**.)

Connect the shift register clock to KEY[3] and use the momentary key to cycle through the first ten values of the PRBS. The random output should be displayed as two hexadecimal digits.

Checkpoint: You should get to this point by the end of the second week.

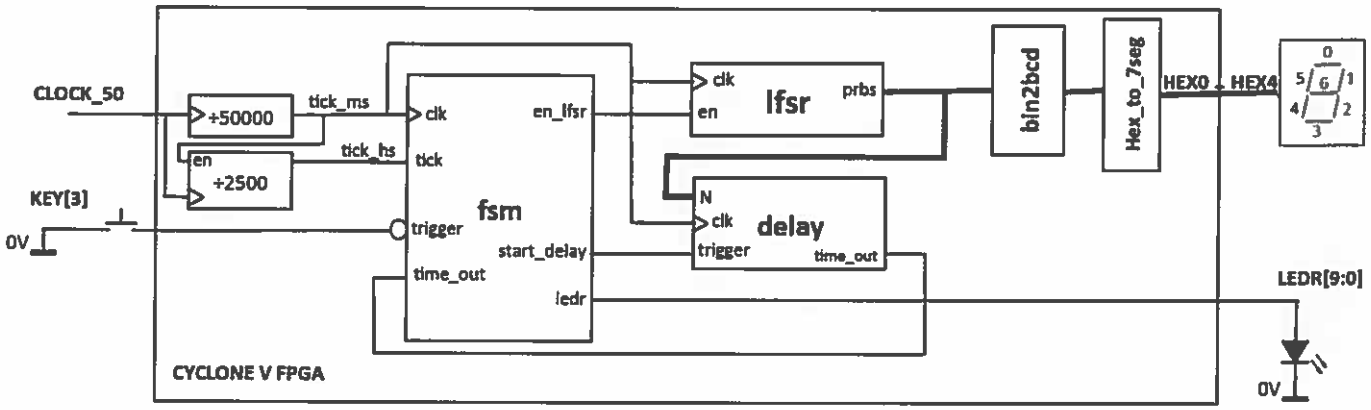
4.0 Experiment 8 (Optional challenge): Starting line delay circuit

The next two experiments are optional. They are designed to provide a challenge to those who finish early, or for those who want to learn more about digital design, Verilog and FPGAs. The two experiments are linked – what you designed in Experiment 8 will be used in Experiment 9.

The goal here is to design a Formula 1 style of race starting lights. The specification of your circuit is:

1. The circuit is triggered (or started) by pressing KEY[3] (don't forget KEY[3] is low active);
2. The 10 LEDs (below the 7-segment displays) will then start lighting up from left to right at 0.5 second interval, until all LEDs are ON;
3. The circuit then waits for a random period of time between 0.25 and 16 seconds before all LEDs turn OFF;
4. You should also display the random delay period in milliseconds on five 7-segment displays.

In order to assist you in designing this circuit without spending too much time, the following overall block diagram of the circuit is provided. You should also download the solution bit-stream for this experiment from the experiment webpage (**ex8sol.sof**) and try it out before attempt it yourself.



In the above diagram, all signals on the left of the block are inputs and the signals on the right are outputs.

The two clock divider circuits provide clock ticks once every 1ms and 0.5sec respectively. Each clock tick should be a positive pulse lasting one period of **CLOCK_50** (i.e. 20ns). The system then use the **tick_ms** signal as the clock of the remaining circuit.

The **LFSR** module produces a pseudo-random binary sequence (**PRBS**), which is used to determine the random delay required. The **enable** signal to the **LFSR** allows this to cycle through a number of clock cycles before it is stopped at a random value.

The **delay** module is triggered after all 10 LEDs are lid, and then provides a delay of **N** clock cycles (at 1ms period) before asserting the **time_out** signal (for 1ms).

The delay value N is fed to the binary to BCD converter, which then drives the 7-segment displays.

There are a number of design decisions to be made:

1. How many bits LFSR is required?
2. How many bits should you use in the delay module?

The FSM module is the key module to the entire system. You have to decide what are the states that are required, draw the state diagram and then map that to Verilog.

5.0 Experiment 9 (Optional challenge): A Reaction Meter

Extend your circuit in Experiment 8 by adding a reaction counter. This should count the time between all the LEDs turning OFF and you pressing **KEY[0]**. The reaction time, instead of the random delay, should be displayed on the 7-segment displays in milliseconds.

Department of Electrical & Electronic Engineering

Imperial College London

2nd Year Laboratory

Experiment VERI: FPGA Design with Verilog (Part 3)

PART 3 – Analogue I/O and SPI serial Interface

1.0 The Add-on Card

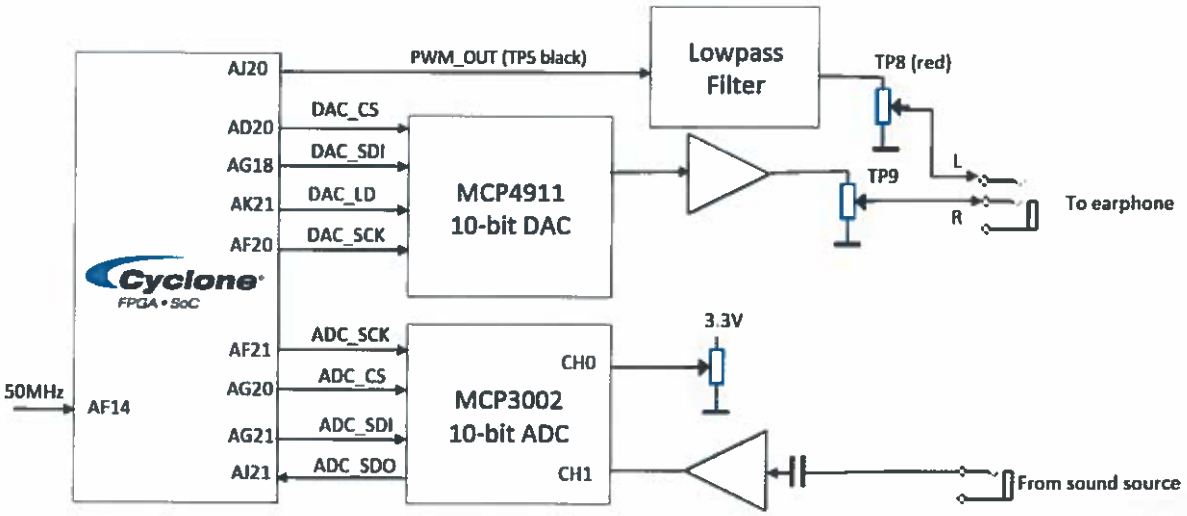
This part of the experiment introduces you to the add-on card to the DE1 board. The add-on card consists of a 10-bit ADC and a 10-bit DAC, a quad op-amp, sockets for earphone (analogue output) and sound source (analogue input), and a potentiometer. The overall block diagram of the add-on card is shown below. It should be plugged into the expansion socket furthest away from the edge of the DE1 board. Beware of the alignment between the plug and the socket. If the add-on board is inserted correctly, the green LED will light up when the DE1 board is turned ON.

You do not need to understand all the circuitry on this board in details. Nevertheless, the schematic diagram and a detail explanation on how this board works, together with all the datasheets of the components used, are provided on the Experiment webpage.

For this part of the experiment, you would need to bring your personal earphone, and from the Lab, get a 3.5mm lead and a digital voltmeter.

By the end of this part of the experiment, you will have:

- Understood and verified the operation of the Serial-to-Parallel Interface (SPI) of the digital-to-analogue converter (DAC) MCP4911 using Modelsim;
- Tested the DAC and measured its output voltage range;
- Learned how to use a ROM (read-only memory) and a constant coefficient multiplier;
- Use the analogue-to-digital converter (ADC) MCP3002 to convert dc voltages;
- (Finally), designed a sinewave tone generator with variable frequency which is controlled with the slide switches, and the frequency value showed on the 7-segment displays in decimal format.



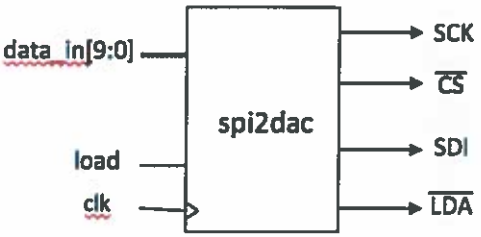
2.0 Experiment 10: Interface with the MCP4911 Digital-to-Analogue Converter

Step 1: Understanding Datasheet - Go to the Experiment website and download the datasheet for the MCP4911 DAC and the file **spi2dac.v**, which is a Verilog module that implements the SPI interface circuit to communicate with the DAC. Make sure that you understand from reading the datasheet:

- the purpose of each pin on the DAC (Section 3.0, page 17 of datasheet);
- how information is sent to the DAC through the serial data input (SDI) pin (Section 5.0, page 23-24);
- how to configure the DAC's internal function (page 25);
- DAC's timing specifications and timing diagram (pages 4 and 7).

There is no need for you to know how exactly the DAC works internally. However, you need to have sufficient appreciation of the serial interface in order to conduct this part of the experiment. Furthermore, don't worry if you don't fully understand the Verilog code in **spi2dac.v**. This will be explained in a Lecture.

Step 2: Timing diagram – The **spi2dac** module takes a 10-bit number in parallel (controlled through the load signal which must be high for at least 20ns) and generates the necessary serial signals to drive the MCP4911 DAC. Based on the information from the Datasheet, draw in your logbook the expected timing diagram of the SPI interface signals when a word 10'h23b is sent to the DAC.

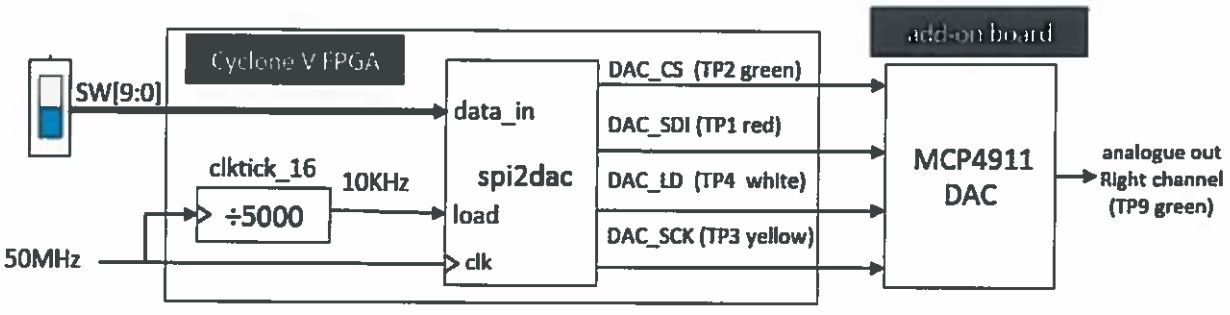


Step 3: Verify timing of spi2dac.v using Modelsim – The steps are:

1. Create a project ex10 and a top-level module ex10_top.v
2. Copy to the directory ex10 the file spi2dac.v downloaded from the webpage
3. Make this file top-level module (for now)
4. Click: ... **> Process > Start > Analyze and Synthesise**
5. Start Modelsim **(Tools > Run Simulation Tools > RTL Simulation)**
6. Design a **do-file** as a testbench to exercise the input signals correctly
7. Run the do-file and match the waveform generated with your prediction

Step 4: Testing the DAC on DE1

In order to test the **spi2dac.v** module and verify that DAC works properly, create the top-level design **ex10_top.v** that implements the circuit shown in the following diagram.



The **data_in** value determined by the 10 switches (**SW[9:0]**) is loaded to the **spi2dac** module at a rate of 10k samples per second as governed by the **load** signal. The steps for this part are:

1. Download from the experiment website the file **spi2dac.v**.
2. Check that the **clktick_16.v** module that you used last week is in the **"mylib"** folder.
3. Create a top-level module **ex10_top**.v to connect all modules together as shown in the diagram.
4. Click: **Project > Add/Remove Files in Project** ..., and select all the relevant files used here. This step is important – it allows you to select which modules to include in your design.
5. When **ex1__top**.v is the current file, click: **Project > Set as Top-Level Entity**. This is another useful step, which defines the top module, and all those module below this one, for compilation. With steps 4 and 5, you can move up or down the design hierarchy in a project for compilation.
6. Edit the **ex10_top.qsf** file to include **pin_assignment.txt**.
7. Compile and correct errors as necessary.

Once the design is compiled without error, download the bit-stream file to the DE1 board. Using the DVM, measure the DAC output voltage at TP9 for SW[9:0] = 0 and 10'h3ff. (The voltage range of the DAC output should be from 0V to 3.3V.)

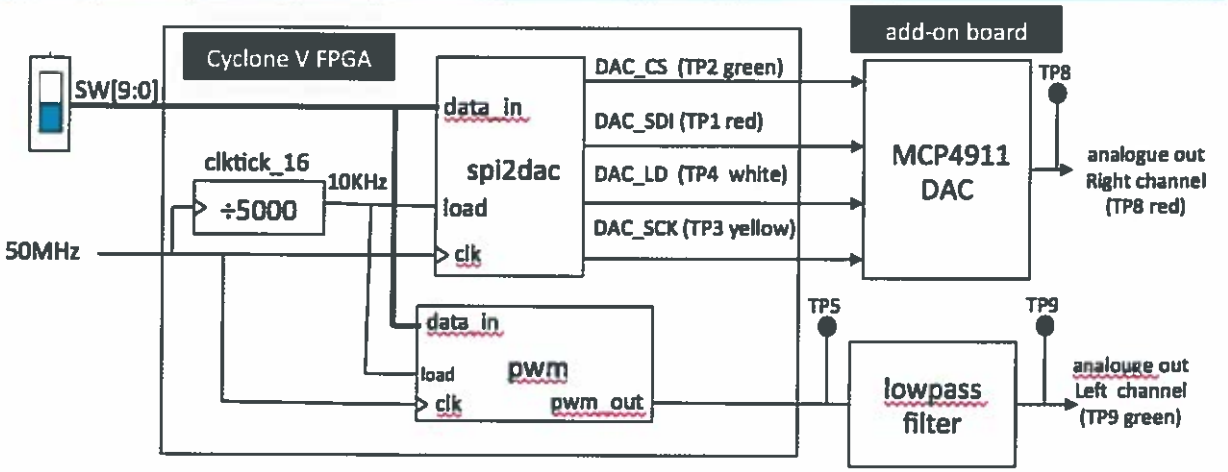
Step 5: Verify the signals on an oscilloscope

Confirm that the signals produced by the FPGA with the **spi2dac.v** module agree with those from Modelsim. Set SW[9:0] to 10'h23b and measure DAC_SCK (TP3) and DAC_SDI (TP1) using an oscilloscope. You may need to trigger the scope externally with the DAC_CS signal (TP2). Compare the waveforms to those predicted by Modelsim.

3.0 Experiment 11: D-to-A conversion using pulse-width modulation

Instead of using a DAC chip (and SPI serial interface to communicate with the chip), an alternative method to produce an analogue output from a digital number is to use pulse-width modulation (PWM). The Verilog code for a **pwm.v** module is given to you in Lecture 9 slide 15.

Create a design **ex11_top.v** according to the circuit shown below. Use the scope to examine the signals at TP5 and TP8. Compare the output voltage ranges at TP8 and TP9.



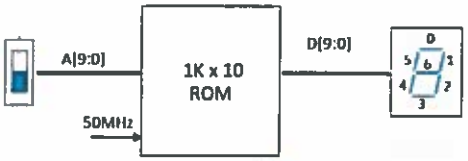
4.0 Experiment 12: Designing and testing a sinewave table in ROM

This part of the experiment leads you through the design of a 1K x 10 bit ROM, which stores a table of sine values suitable to drive our DAC. The relationship between the content of the ROM D[9:0] and its address A[9:0] is:

$$D[9:0] = \text{int}(511*\sin(A[9:0]*2*\pi/1024)+512) \qquad \text{for } 1023 \geq A[9:0] \geq 0$$

Since the DAC accepts an input range of 0 to 1023, we must add an offset of 512 in this equation. (This number representation is know as **off-set binary** code.)

Before generating the ROM in Quartus using the "**Memory Compiler**" tool, we need to first create a text file specifying the contents of the ROM. This can be done in different ways. Included on the Experiment webpage are: 1) a Python script to do this; 2) a Matlab script to do the same thing; 3) a memory initialization file **rom_data.mif** created by either method. Download these files and examine them.




```
module ROM (
    address,
    clock,
    q);

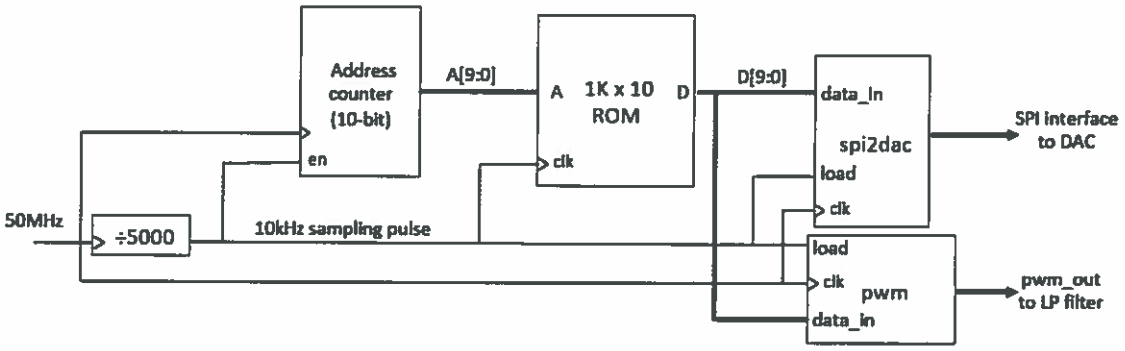
    input   [9:0]  address;
    input          clock;
    output  [9:0]  q;
```

Click Tools > IP Catalog to bring up a tool which helps to create a 1-Port ROM. A catalog window will pop up. Select from the window **>Library >Basic Functions > Onchip Memory > ROM 1-Port.** Complete the on-screen form to create **ROM.v**.

To verify the ROM, create the design **ex12_top.v**, that uses the switches **SW[9:0]** to specify the address to the ROM, and display the contents stored at the specified location on the four 7-segment display. Once this is done and loaded onto the DE1, verify that the contents stored in the ROM matches those specified in the **rom_data.mif** file.

5.0 Experiment 13: A fixed frequency sinewave generator

Let us now replace the slide switches with a 10-bit binary counter and connect the ROM data output to **spi2dac** and **pwm** modules as shown in the figure below. Since the ROM contains one cycle of sinewave and the address to the ROM is incremented every cycle of the 10kHz clock, a perfect sinewave is produced at the left and right outputs of the 3.5mm jack socket.



Implement this circuit and verify that the signals produced by both the DAC and the PWM are as expected. What is the frequency of the sinewave?

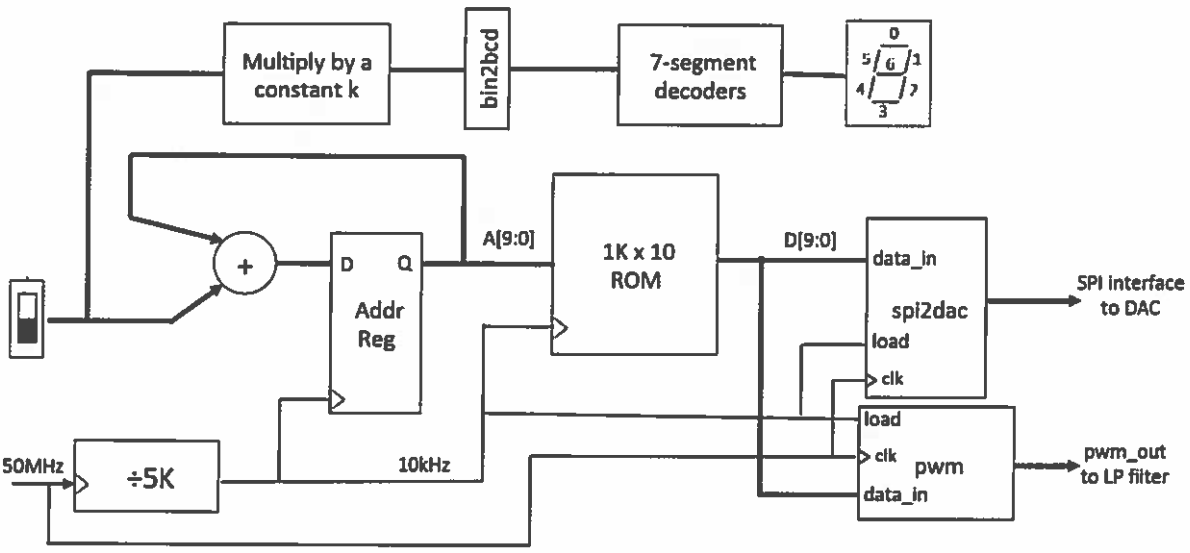
6.0 Experiment 14 (optional challenge): A variable sinewave generator

Combine everything together to produce a design **ex14_top.v**, which produces a variable frequency sinewave using table-lookup method. The sampling frequency is 10kHz, and the sine value is read from the ROM that is preloaded with one-cycle of a sinewave (i.e. the address of the ROM is the phase and the content is the sine value). On every sample period, advance the address (i.e. the phase) by an amount determined by SW[9:0]. Derive the relationship between the output signal frequency and the switch setting.

The overall block diagram is shown below. The switch setting is multiplied by a constant k to convert the phase increment SW[9:0] to frequency.

You can produce a 10-bit x 14-bit constant coefficient multiplier using the IP catalog tool. The 14-bit constant is 14'h2710 (which is 14'd10000). The product is a 24-bit number, and the frequency is the top 14-bits (Why?). The frequency can then be displayed on the 7-segment displays.

Produce a 439Hz sinewave, which is close to 440Hz, the frequency commonly found in tuning forks. Make sure that this is indeed correct (through listening or measuring with a frequency counter).



7. Experiment 15 (Optional Challenge): Using the A-to-D converter

In this experiment, you will learn to use A-to-D converter MCP3002 on the add-on board to convert analogue voltages to digital signals. Again, download from the webpage the **spi2adc.v** module, which is already written for you to use.

Instead of using the slide switches to control the frequency, use the A-to-D converter to convert the dc voltage of the potentiometer (which is between 0v and 3.3v) and use this converter value instead.

To help you know what you should aim for, the solutions for **ex14sol.sof** and **ex15sol.sof** are available to download.

Department of Electrical & Electronic Engineering

Imperial College London

2nd Year Laboratory

Experiment: FPGA Design with Verilog (Part 4)

PART 4 – Real-time Audio Signal Processing

1.0 Putting everything together

In this part of the experiment, you will learn to combine the ADC with the DAC on the Add-on card, and use the DE1 to perform some simple audio processing.

The goal of the final week's laboratory session is to implement a **speech echo effect synthesizer**. You need to bring your earphone to the lab in order to listen to the audio output.

2.0 Experiment 16: An audio in-and-out (all pass) loop

Download from the Experiment webpage the file ex16_proto.zip, which contains the prototype folder for this experiment.

- Examine the contents within this folder. You should find the following Verilog files:

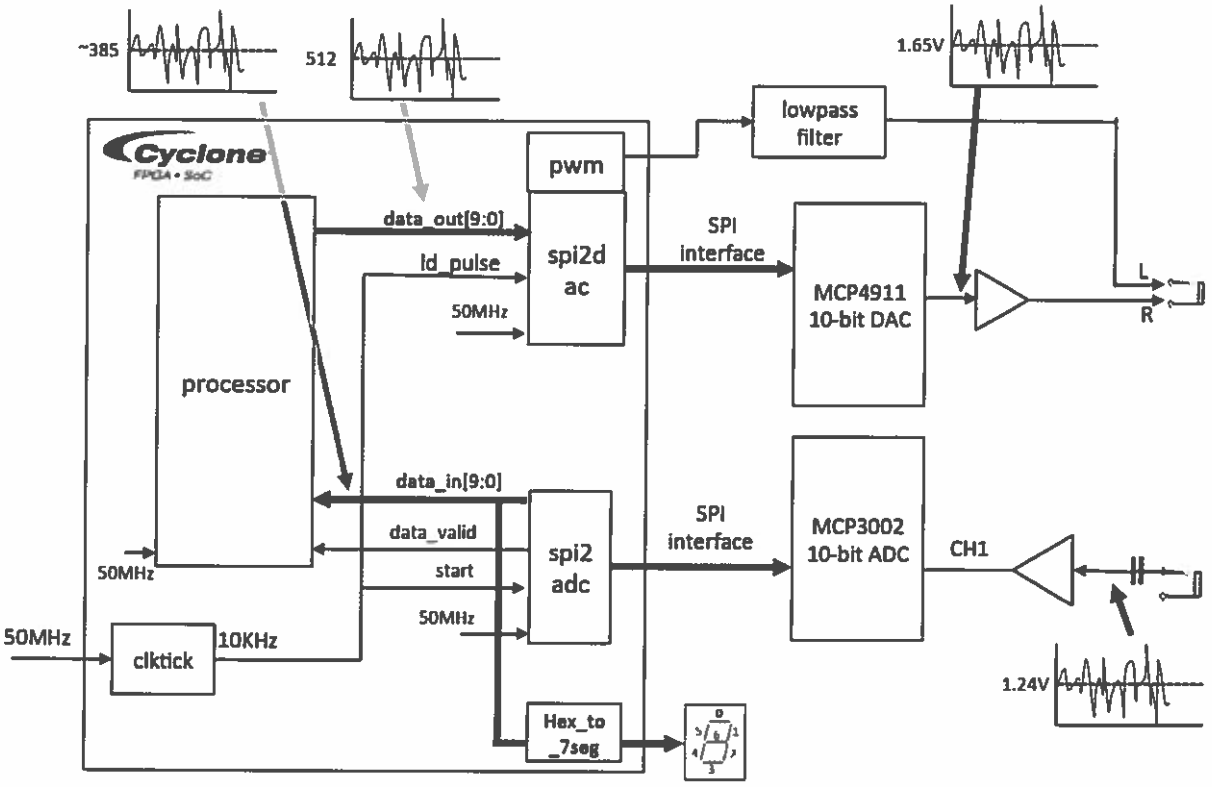
Module	Function
ex16_top.v	Top-level design; interface to pins
spi2dac.v	SPI interface circuit to DAC from Part 3
spi2adc.v	SPI interface circuit to ADC
pwm.v	Pulse-width modulation DAC from Part 3
clktick_16.v	Clock divider to generate sampling clock ticks at 10kHz from Part 2
Pulse_gen.v	Generate a one-cycle pulse on rising edge of a trigger signal (new)
hex_to_7seg.v	Hex to 7-segment decoder from Part 1
allpass.v	"**processor**" module – this performs processing, which is passing input to output for now.

- Study **ex16_top.v**. This specifies a system as shown in the following diagram (the part inside the Cyclone V). Make sure you understand how this works.
- Note in particular how the **spi2adc.v** module is used. Explicitly associating the signal names INSIDE the module to OUTSIDE allow connections to be defined independent of the order. This is a more verbose but is much safer way in making connects to modules.

```
spi2adc SPI_ADC (
    .sysclk (CLOCK_50),
    .channel (1'b1),
    .start (tick_10k),
    .data_from_adc (data_in),
    .data_valid (data_valid),
    .sdata_to_adc (ADC_SDI),
    .adc_cs (ADC_CS),
    .adc_sck (ADC_SCK),
    .sdata_from_adc (ADC_SDO));
```

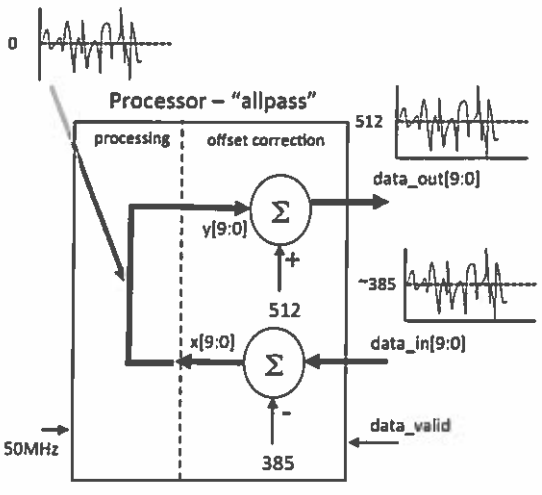
- The ADC has two analogue input channels: CH0 and CH1. They connected to the potentiometer and to the 3.5mm socket respectively. We always use CH1 for **ex16**.
- Now examine the module **allpass.v**. The name of this module is "**processor**" and is different from the name of the Verilog file. There is no need to use the same name except that normally it is more convenient to do so. However, in this case, we have deliberately used the filename "**allpass**" to describe its function, while using a more universal name for the module. You can choose "**allpass.v**" as the source of the module "**processor**" now. Later, you can have a different Verilog file to define a

different "**processor**". Which version of "**processor**" you use in your design is specified in **Project > Add/Remove File in Project.**



- Make sure that you understand fully what the Verilog file "**allpass.v**" does. It actually does very little. It:
    1. Corrects the ADC converter data (which uses offset binary with 0V represented by a value of ~385), but subtracting the offset from data_out[9:0] to obtain a 2's complement value x[9:0].
    2. Connects X to Y, i.e. does nothing and hence "allpass".
    3. Converts the Y value from 2's complement to offset binary for the DAC. The offset now is at 512 as shown below.

- Build your design for testing on the DE1 Board. To do this, you should:
    1. Open each .v file, and use **Processing > Analyze Current File** on each of the Verilog file to ensure that there is not syntax error.
    2. Use **Project > Add/Remove** File in Project to include all the .v files you need. Here we select **allpass.v** to supply the "**processor**" module. In the future, you could substitute **allpass.v** with another file for a different processor.
    3. While **ex16_top.v** is the current file in the editor window, use **Project > Set as Top-level Entity** to define top is the top-level



module.

4. Use **Project > Start > Analysis and Synthesize ...** to check for errors and warnings without compiling everything.
5. Check that Device, Pin and **TimeQuest** clock period are all assigned correctly.
6. Compile the whole design and download the bit-stream file **"ex16_top.sof"** to DE1.
7. Test that it is working properly. You can use the PC to play some music or speech files (downloadable from Experiment webpage), and use an earphone to listen to the DAC output. When no signal is sent to the DE1 board, the display should show a hex value of 181 to 188.

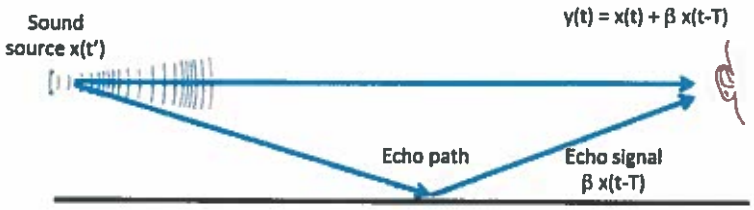
When you get to this part, the experiment framework is shown to be working. It takes audio samples at 10kHz from the ADC, passes it through a processor module and output the processed sample to the DAC.

Test yourself

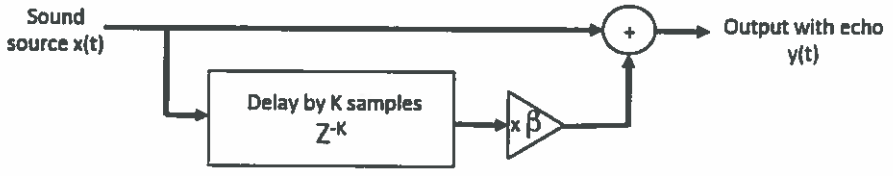
Now create a new Verilog file **mult4.v** which is a processor module (i.e. module name is still "processor"), that amplifies the input by a factor of four. Test that this is working (i.e. the signal to the earphone should be louder or distorted).

3.0 Experiment 17: Echo Synthesizer with fixed delay

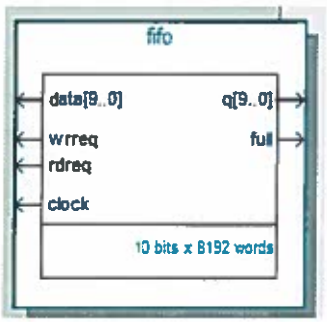
In this part of the experiment, you will design, implement and test a circuit that simulates the effect of simple echo. The diagram below shows two components of a sound source reaching its listener: the direct path signal x(t) and the echo signal β x(t-T) which is a weaker version of x(t) attenuated by a factor β, bounced off the floor. The echo signal is also delayed by T relative to the direct-path signal x(t).



Such simple echo can be implemented as signal flow graph as shown below. This involves three components: a delay block that delays x(t) by K sample periods; a gain block which multiplies the delayed signal by the factor β; and the adder.



The delay block can be implemented with a first-in-first-out (FIFO) buffer. A FIFO is found in all forms of digital systems. The rule is simple: receiving data are stored in sequence in such a way that they can be retrieved in the order that they arrive. When a new data item arrives and the FIFO is not full, it is written to the FIFO. As a stored data item is retrieved, it is removed from the FIFO. This allows the send and retrieve rates to be different in the short term.



If the send rate is higher than retrieve rate, eventually the buffer will get full. If the buffer is full, it should not receive any more data (otherwise existing store data would be corrupted). A "full" status signal is asserted to tell the sender not send any more data. Similarly if the buffer is empty, it cannot provide any data for retrieval. An "empty" status signal is used to indicate that the FIFO has no more data to provide.

Create a new project using the files from Experiment 12 as your prototype. With IP Catalog tool, generate a FIFO component of size 8192 x 10-bit as shown here. You only need to provide only the "full" status signal. This FIFO is used to store the most recent 8192 samples, hence providing a delay of 0.8192msec since the sampling frequency is 10KHz. Before the echo simulation circuit starts to provide the echo, the FIFO must first be completely filled (i.e. wait until the "full" signal is asserted). Thereafter, the writing of the ADC sample and DAC sample is synchronous, and the FIFO remains full. The read data is always the write data delayed by 8192 sample period.

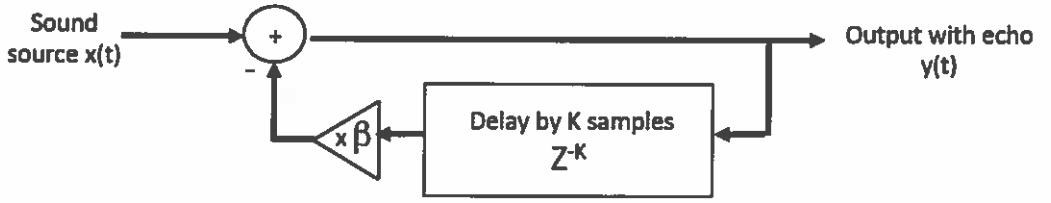
The attenuation factor β should be ½ or ¼, which can easily be implemented with a simply binary shift.

Deliverable

Implement the simple echo simulator and test that it works. For the purpose of test, download three different sound files: clapping.mp3, hello.mp3 and hitchhiker.mp3, and play them on the PC or phone in a loop. Use your earphone to listen to the effect of the echo synthesizer.

4.0 Experiment 18: Multiple echoes

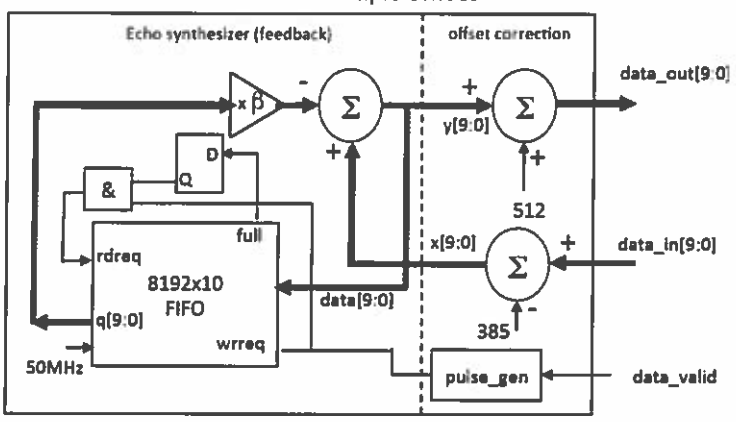
The design in Experiment 13 produces a single echo. The signal flow graph only has feedforward paths. Multiple echoes can be produce with a slight modification of the signal flow graph to the one shown below.



The delay block now stores the output sample y(t) instead of the input sample x(t). The attenuated and delayed y(t) is SUBTRACTED from x(t) to produce the next output. (Why must this be a subtract and not an add?)

Provide a design to implement this architecture and test it.

Processor – multiple echoes

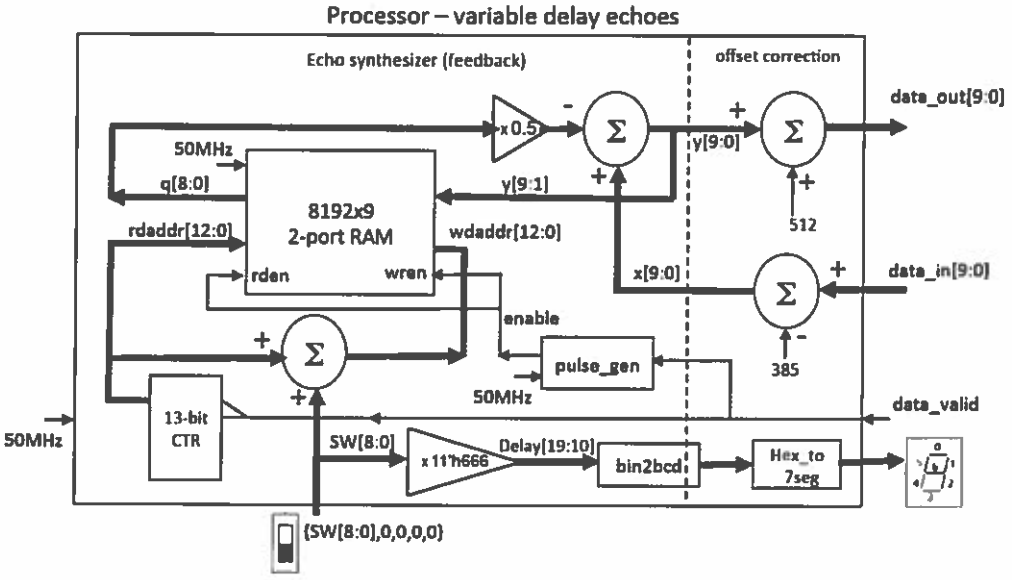


5.0 Experiment 19 (Optional challenge): Echo Synthesizer with Variable delay

In this experiment, you will design, implement and test a system with variable delay. A bit-stream (echo.sof) that implements a solution can be downloaded from the Experiment webpage. You also need to download the three MP3 test files. Connect the audio input to the speaker of the PC and play the audio files in a loop. Program DE1 with echo.sof and listen to the output with your earphone. Change the delay of the echo with SW[8:0]. The amount of delay in millisecond is displayed on the 7-segment displays as a decimal number.

The design of this experiment is shown in the block diagram below. It consists of a number of modules:

- RAM Delay Bock - In place of the FIFO to implement the delay block, it uses a 2-port RAM block (8192 x 9-bit) – one write port (to store the ADC samples) and one read port.
- Address Generator - A 13-bit counter is used to generate the read address to the RAM. (Why 13-bits?) The counter value is incremented on the negative edge of the data_valid signal at a frequency of 10KHz. In this way, the address generator computes the address used on the next read and write cycle. The write address is generated from the read address by adding the value taken from SW[8:0]. Since the address is 13-bits wide, the 9-bit delay value is zero-padded in its lower 4 bits. Therefore, the delay between the read and write samples is: SW[8:0] x 16 x 0.1 msec.
- The read and write enable signals are common, and it is generated from the data_valid signal with the pulse_gen module.
- The write data value y[9:1] is 9-bit instead of 10-bit wide. This is because the embedded memory in the Cyclone III FPGA is configurable as 9-bit in data width, but not 10-bit. Therefore the output data value is truncated to 9-bit before storing in the delay block.
- The read data value is of course also 9-bit wide. Therefore the x0.5 can easily be implemented by sign-extending the 9-bit value to 10-bit: {q[8],q[8:0]}.
- The implementation of the feedback loop to generate the echo effect is identical to that from the previous experiment.
- To display the delay value in milliseconds, the value of SW[8:0] is first multiplied by 1638 (why) with a constant multiplier. This gives a 20-bit product, the most significant 10-bits of which is the delay in milliseconds. (Why?) This is then converted from binary to BCD and decoded for display on the 7-segment displays.

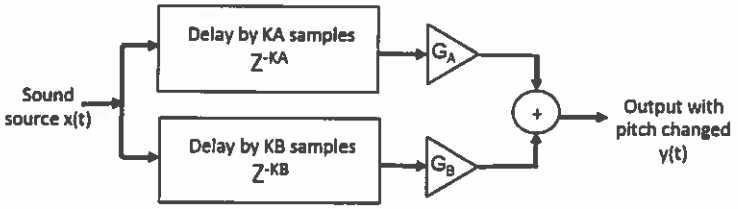


6.0 Experiment 20: Voice Corruptor (Not part of this Lab)

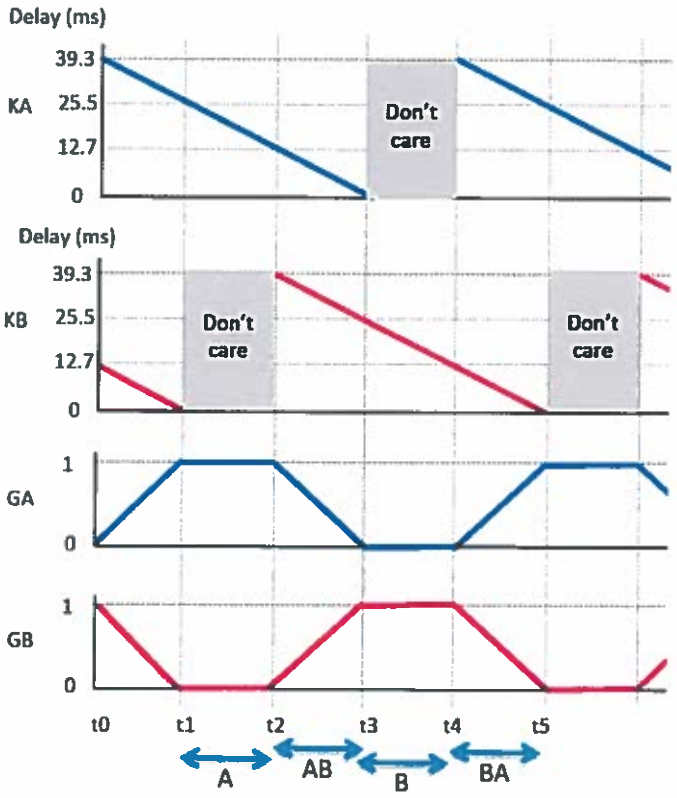
This part of the experiment is outside the scope of the experiment. It is designed to provide you with an open problem so that you can explore designing digital systems and implementing digital circuits using the DE1 and the add-on card at your own leisure. You need to check out a set of kits to take home from stores. For example, you might want to try this out over the Christmas break.

You are now equipped with all the tools and knowledge to design a reasonably complex audio processing system. The idea here is to design something that will take a human speech signal and then "corrupt" in a way that the identity of the speaker is masked while the speech remain intelligible.

One way to do this is to change the pitch of the speaker (e.g. make it sounds like Donald Duck). There are many ways to perform pitch change of speech. One method, which is linked to the previous experiments, is to employ a technique based on cross fading (i.e. combining) of two separately delayed version of the speech signal. The technique is depicted in the block diagram below.



The sound source is delayed through two separate blocks, providing KA and KB sample delays, which vary with time. The delayed signals are then attenuated by GA and GB, and combined with the adder. In order to minimize the artifacts and discontinuities in the output signal and to maintain a constant volume, the gain values GA and GB are designed to cross fade with each other – i.e. when one is ramping up (from 0 to 1), the other is ramping down. A plot of the four parameters, KA, KB, GA and GB, vs time is shown below.



There are four regions.

1. Region A (t1 to t2) - Only channel A is contributing to the output. The delay KA is gradually decreasing linearly from 25.5ms to 12.7ms (255 to 127 x 100μs). The gain GA is constant at 1.
2. Region AB (t2 to t3) – Both channels contribute to the output with A decreasing and B increasing their respective contributions. The two channels are cross faded before GA drops from 1 to 0 while GB increases in the other direction.
3. Region B (t3 to t4) – This is similar to Region A, but the behavior applies to channel B instead of channel A.
4. Regions BA (t4 to t5) – Similar to Region AB, but the two channels are reversed.

The pattern repeats itself indefinitely. Note that the "don't care" portion of KA and KB is due to the fact that during this period, the gain GA or GB is zero.

Hints:

- Initially, try the delay ramping gradient of 0.5, i.e. the delay is dropped by k over time 2*k.
- You can use a 9-bit down counter to define both the delay KA and the four regions.
- You can derive all other values: KB, GA and GB, from the counter values.
- You can design a four state synchronous state machine to control the corruptor circuit.
- Instead of delay varying ramping high to 0, you can reverse the direction of the ramping. Alternative you can design the delay to vary up and then down.
- In addition to pitch changes, you may explore other audio effects.